



SRS — AI Text-to-SQL Data Analyst System



Purpose: Build an AI-powered analyst dashboard that converts natural language to SQL, executes queries, returns results, generates insights, and auto-builds reports.



MVP success criteria

- Response time < 3s for common queries
- Safe execution (SELECT-only by default)
- Explain results باللهجة المصرية
- Export a shareable report



Reading guide

- Start with: Execution Flow (Section 9)
- Then: Functional Requirements (Section 4)
- Then: Architecture (Section 6)

1) Introduction

[1.1 Purpose](#)

[1.2 Scope](#)

[1.3 Definitions](#)

2) Overall Description

[2.1 Product Perspective](#)

[2.2 User Personas](#)

[2.3 System Overview Flow](#)

- 3) System Features
- 4) Functional Requirements (FR)
- 5) Non-Functional Requirements (NFR)
- 6) System Architecture
 - High-level
 - Core agents/services
- 7) Detailed Components
- 8) Data Architecture
- 9) Execution Flow
- 10) Visualization System
- 11) Documentation System
- 12) Storage
- 13) Security
- 14) Future Enhancements

▼ ✨ Demo scenario (end-to-end example)

- Question: "What were total sales by month in 2025?"
- Steps:
 - Retrieve relevant schema
 - Generate SQL
 - Validate + optimize
 - Execute safely
 - Return results + chart + explanation
- Output:
 - SQL generated + validated
 - Results table (paginated)
 - Auto-selected chart (line chart)
 - 3–5 bullet insights in Egyptian Arabic

1) Introduction

1.1 Purpose

The user can:

- Ask questions in natural language
- Get an SQL query
- View data results
- Understand insights (explained in Egyptian Arabic)
- Generate reports automatically

1.2 Scope

Includes:

- Frontend dashboard
- Backend API
- AI multi-agent system
- Data source integration (CSV + databases)
- Visualization engine
- Documentation/report generator

1.3 Definitions

- **Text-to-SQL:** Converting natural language questions into SQL
 - **Task:** Intelligent unit handled by an agent
 - **Schema:** Data structure (tables + columns)
 - **Insight:** Inference derived from data
 - **Semantic cache:** Similar-query cache for re-use
-

2) Overall Description

2.1 Product Perspective

The system acts as:

- AI Analyst

- BI Tool
- Data Integration Platform
- Reporting System

2.2 User Personas

- **Business user:** wants fast insights
- **Developer:** wants ready-to-run SQL
- **Data analyst:** wants deeper analysis

2.3 System Overview Flow

User → UI → Backend → AI Agents → Database → Results → UI

3) System Features

▼ 3.1 Query & UX

- Natural language input
- "Generate" action
- Voice input (future)
- Query history and re-run

▼ 3.2 SQL Generation & Reliability

- Accurate SQL generation
- Self-healing (retry + fix)
- Context-aware schema selection

▼ 3.3 Results & Insights

- Results table with pagination + sorting
- Insight generation (summary + key findings)
- Explanation engine (Egyptian Arabic, context-aware)

▼ 3.4 Visualization

- Bar / line / pie charts
- Auto-selection based on the result shape

▼ 3.5 Data Sources

- CSV upload → becomes tables
- Database connections
- Multi data source query (JOIN across CSV + DB)
- Schema understanding (column meaning mapping)

▼ 3.6 Reporting

- Documentation/report generator (overview, questions, SQL, insights, recommendations)

4) Functional Requirements (FR)

FR	Name	Description
FR1	Upload CSV	User uploads a file; it is ingested and available as a queryable table.
FR2	Connect Database	User enters credentials; system tests and registers the connection.
FR3	Retrieve Schema	System extracts tables + columns (and relationships when available).
FR4	Context Filtering	System selects relevant tables/columns for the question to reduce prompt size and improve accuracy.
FR5	Generate SQL	Produce SQL from the user question.
FR6	Optimize Query	Avoid full scans where possible; add LIMIT defaults; prefer indexed filters when known.
FR7	Execute Query	Run SQL safely and return results with pagination support.
FR8	Self-Healing	If execution fails, generate a fix and retry within safe limits.
FR9	Generate Insights	Analyze results and produce key findings.

FR	Name	Description
FR10	Visualization	Create appropriate charts based on result columns.
FR11	Suggestions	Generate follow-up questions that deepen analysis.
FR12	Documentation	Generate a structured report.

5) Non-Functional Requirements (NFR)



Security

- Encrypt credentials
- Query sandboxing
- Rate limiting
- Default policy: SELECT-only unless explicitly enabled



Performance & scalability

- Typical response < 3 seconds
- Support multiple users (scalable workers)
- Cache schema and frequent queries when possible



Usability & reliability

- Simple, clear UI
- Self-healing execution for common SQL errors
- Clear error messages and "what to ask next" hints

6) System Architecture

High-level

- Frontend (React)
- Backend API (FastAPI)
- LangGraph orchestrator
- Unified database layer (PostgreSQL)

Core agents/services

- Context retriever
 - Semantic cache
 - SQL generator (LLM)
 - SQL optimizer
 - Query validator
 - Execution engine
 - Self-healing loop
 - Insight generator
 - Visualization agent (Python)
 - Suggestion engine
-

7) Detailed Components

▼ 7.1 Frontend

Tech:

- React / Next.js
- Tailwind

Modules:

- Dashboard
- Data sources

- History
- Documentation

▼ 7.2 Backend

Tech:

- FastAPI

Services:

- Query service
- Data source service
- Schema service
- Documentation service

▼ 7.3 AI Layer (Agents)

1. Context retriever
2. Semantic cache (FAISS / Pinecone)
3. SQL generator (LLM)
4. SQL optimizer
5. Validator
6. Execution engine
7. Self-healing
8. Insight generator
9. Visualization agent
10. Suggestion engine

8) Data Architecture

- Unified data layer
- PostgreSQL as main store
- CSV handling: ingest to tables

9) Execution Flow

1. User enters question
 2. Retrieve schema
 3. Check semantic cache
 4. Generate SQL
 5. Optimize
 6. Validate
 7. Execute
 8. Self-heal if error
 9. Generate insights
 10. Visualize
 11. Suggest next queries
-

10) Visualization System

Tools:

- Pandas
- Matplotlib

Output:

- Chart image(s)
-

11) Documentation System

Report includes:

- Overview
- Questions
- SQL

- Insights
 - Recommendations
-

12) Storage

Options:

- PostgreSQL
 - Vector DB
-

13) Security

- Credential encryption
 - Query sandboxing
 - Rate limiting
-

14) Future Enhancements

- Voice queries
- Real-time streaming
- Team collaboration
- Auto dashboards